



Documentação Técnica Completa

Versão 3.0 — Fase 3A

■ Data	Março 2026
■ ■ Stack	Hono + TypeScript + Cloudflare D1 (SQLite) + Wrangler Pages
■ Frontend	9.816 linhas de JavaScript puro (SPA)
■ Backend	30 rotas de API em TypeScript
■ ■ Banco	35 tabelas relacionais em Cloudflare D1
■ Versão	v3.0 — 5 novas funcionalidades (Fase 3A)

Este documento cobre 35 capítulos, incluindo arquitetura, banco de dados, sistema de planos, autenticação, todos os módulos funcionais, mapa de interdependências e referência completa de endpoints.

. ÍNDICE

--	--

1. VISÃO GERAL DA ARQUITETURA

```
Browser (HTML/CSS/JS puro – app.js 9.816 linhas)
■
■ HTTP / Fetch API
▼
Cloudflare Pages (edge global)
■
■ Hono Router (src/index.tsx)
▼
30 Rotas de API (/api/*)
■
■ Cloudflare D1 (SQLite distribuído)
▼
35 Tabelas relacionais
```

Arquivos principais

Arquivo	Função
src/index.tsx	Entry point — registra todas as 30 rotas, CORS, health check, Service Worker
src/routes/*.ts	30 módulos de rota (um por funcionalidade)
src/lib/auth.ts	Helpers: hash bcrypt, geração de token, verificação de senha
src/routes/planos.ts	Definição de limites FREE/PREMIUM/PRO — consultado por todas as rotas
public/static/app.js	SPA completa: navegação, modais, chamadas API, renderização de páginas
migrations/*.sql	11 arquivos de schema cumulativos (0001 → 0011)
wrangler.jsonc	Binding do banco DB → verdemais-production
ecosystem.config.cjs	PM2 config para wrangler pages dev em sandbox

Padrão de autenticação

Toda rota protegida usa o middleware `requireAuth` de `src/routes/auth.ts`:

```
Header: Authorization: Bearer <token>
OU Cookie: vm_token=<token>
```

O middleware valida o token na tabela `sessions` e injeta `c.get('user')` com `{ id, nome, email, plano, perfil_investidor }` para uso em qualquer handler.

2. BANCO DE DADOS — TODAS AS TABELAS

Tabelas Core (criadas em 0001_initial_schema.sql)

Tabela	Propósito
users	Cadastro de usuários
receitas	Entradas financeiras
despesas	Saídas financeiras
metas	Objetivos financeiros com prazo e valor
investimentos	Portfólio de investimentos
assinaturas	Controle de plano ativo do usuário
sessions	Tokens de autenticação com expiração

Tabelas v2 (0002 → 0010)

Tabela	Propósito
cartoes	Cadastro de cartões de crédito
card_charges	Lançamentos de cartão organizados por fatura
financiamentos	Financiamentos imobiliários/veicular com parcelas
emprestimos	Empréstimos pessoais/consignados
lembretes	Contas recorrentes a pagar
lembretes_historico	Histórico de ações dos lembretes
conquistas_definicoes	Catálogo global de conquistas (seed)
conquistas_usuario	Conquistas desbloqueadas por usuário
ia_insights	Cache de análises geradas pela IA
orcamentos	Limites por categoria por mês
recorrencias	Transações automáticas mensais
reserva_emergencia	Reserva de emergência legado (única por usuário)
tags	Tags criadas pelo usuário
despesa_tags	Relação N:N despesa ↔ tag
cdi_historico	Cache do CDI real (BCB)
alertas_cartao	Alertas configurados para limites de cartão
email_verifications	Códigos OTP para verificação de e-mail
pagamentos	Histórico de pagamentos Asaas

Tabelas v3.0 (0011_fase3a_features.sql)

Tabela	Propósito
specialized_reserves	Múltiplas reservas por objetivo
reserve_transactions	Depósitos/saques em cada reserva
detected_subscriptions	Assinaturas detectadas pelo algoritmo
weekly_challenges	Controle do Desafio 52 Semanas
shared_expenses	Divisão de despesas (Modo Casal)
amortization_simulations	Histórico de simulações de amortização

3. SISTEMA DE PLANOS E LIMITES

Arquivo: `src/routes/planos.ts` — consultado por TODAS as rotas que têm limites.

Preços e posicionamento

Plano	Preço	Público
FREE	Grátis	Usuário iniciante — experimenta sem compromisso
PREMIUM	R\$ 19/mês	Usuário engajado — controle total
PRO	R\$ 49/mês	Usuário avançado + acesso à API externa

Tabela de limites por funcionalidade

Funcionalidade	FREE	PREMIUM	PRO
Metas simultâneas	3	Ilimitado	Ilimitado
Cartões de crédito	2	10	Ilimitado
Lembretes ativos	5	Ilimitado	Ilimitado
Investimentos cadastrados	3	Ilimitado	Ilimitado
Empréstimos ativos	2	Ilimitado	Ilimitado
Financiamentos ativos	1	Ilimitado	Ilimitado
Despesas por mês	30	Ilimitado	Ilimitado
Receitas por mês	10	Ilimitado	Ilimitado
Reservas especializadas	1	3	Ilimitado
Score de Saúde Financeira	■	■	■
Insights por IA	■	■	■

Funcionalidade	FREE	PREMIUM	PRO
Relatório anual	■	■	■
Simulação de investimentos	■	■	■
Exportação em PDF	■	■	■
Amortização extraordinária	■	■	■
Projeção financeira	■	■	■
Recorrências automáticas	■	■	■
Acesso à API externa	■	■	■
Reserva de emergência	■	■	■
Conquistas	■	■	■

Como o limite é verificado no backend

Toda rota que tem limite faz a checagem antes de inserir no banco:

```
const lim = getLimites(user.plano) // retorna o objeto de limites do plano
if (lim.metas !== Infinity) {
  const count = await DB.prepare('SELECT COUNT(*) as n FROM metas WHERE
  user_id=?').bind(userId).first()
  if (count.n >= lim.metas)
    return c.json({ error: MSG_UPGRADE.metas, upgrade: true }, 403)
}
```

O frontend trata o `upgrade: true` e exibe o modal de upsell automaticamente.

4. AUTENTICAÇÃO E SESSÕES

Rota: `/api/auth` — `src/routes/auth.ts`

Fluxo completo de cadastro

```
1. Usuário preenche nome + e-mail + senha
2. GET /api/auth/check-email?email=xxx
→ Valida formato, bloqueia domínios temporários (50+ domínios listados)
→ Verifica se e-mail já existe
3. POST /api/auth/register { nome, email, senha }
→ Valida campos (mínimo 3 chars nome, 8 chars senha)
→ Hash da senha via Web Crypto (bcrypt-like)
→ Insere em users com plano='free' e avatar_color aleatório (verde)
→ Cria registro em assinaturas (plano='free', status='ativo')
→ Gera OTP de 6 dígitos, válido 10 minutos → salva em email_verifications
→ Cria sessão temporária (token JWT-like 32 bytes hex)
→ Retorna: { user, token, otp_required: true, _dev_otp: "123456" }
4. POST /api/auth/verify-otp { email, code }
→ Verifica expiração (10 min), tentativas (máx 5), código correto
→ Marca como verificado em email_verifications
5. Usuário já está autenticado com o token retornado no cadastro
```

Fluxo de login

```
POST /api/auth/login { email, senha }
→ Busca usuário pelo e-mail
→ Verifica senha com Web Crypto
→ Atualiza ultimo_acesso
→ Gera novo token de sessão (expira em 30 dias)
→ Retorna { user: { id, nome, email, plano, perfil_investidor, avatar_color }, token }
```

Gerenciamento de sessão

```
GET /api/auth/me → Retorna dados do usuário logado (valida token)
POST /api/auth/logout → Deleta a sessão do banco
POST /api/auth/resent-otp → Reenvia OTP (cooldown de 1 minuto entre envios)
```

Armazenamento no frontend

```
localStorage.setItem('vm_token', token)
localStorage.setItem('vm_user', JSON.stringify(user))
// Enviado em todas as chamadas:
headers: { Authorization: `Bearer ${token}` }
```

Tabelas envolvidas

- `users` → cadastro principal
- `sessions` → tokens ativos (token + expires_at)
- `assinaturas` → plano vinculado ao usuário
- `email_verifications` → OTPs pendentes

5. DASHBOARD — CENTRO DE CONTROLE

Rota: GET /api/dashboard — src/routes/dashboard.ts

O Dashboard é a página principal. Ele **agrega dados de 10 fontes diferentes** em uma única chamada e é o único lugar que calcula o Score de Saúde Financeira.

Dados retornados

Resumo financeiro do mês atual

Campo	Fonte	Lógica
total_receitas	receitas	SUM por mês/ano, usando data
total_despesas	despesas	SUM usando critério duplo: se status='pago' usa data; se pendente usa COALESCE(vencimento, data)
saldo_liquido	Calculado	receitas – despesas
taxa_poupanca	Calculado	(saldo / receitas) × 100
fatura_cartao_mes	card_charges	SUM de charges com billing_month/year = mês atual e status='pendente'
total_parcela_mensal_dividas	Calculado	parcelas empréstimos + parcelas financiamentos + fatura cartão
comprometimento_dividas_pct	Calculado	(parcelas / receitas) × 100

Score de Saúde Financeira (Premium/Pro)

O score começa em 50 e é ajustado por 5 fatores:

Fator	Condição	Pontos
Taxa de poupança	≥ 20%	+20
Taxa de poupança	10–19%	+10
Taxa de poupança	0–9%	0
Taxa de poupança	Negativa	–20
Saldo positivo	Saldo > 0	+5
Investimentos	Tem qualquer investimento	+10
Metas ativas	Tem pelo menos 1 meta	+5
Comprometimento dívidas	> 50% da renda	–25
Comprometimento dívidas	30–50%	–15
Comprometimento dívidas	20–30%	–8
Sem dívidas	Nenhuma dívida ativa	+10

Range final: 0–100 (clamped)

Para plano FREE: `score_saude` retorna `null`, `score_bloqueado`: `true`

Outros dados do Dashboard

- `evolucao` → Gráfico dos últimos 6 meses (receitas, despesas, saldo por mês)
- `categorias_despesas` → Top 8 categorias com maior gasto no mês
- `categorias_receitas` → Top 6 categorias de receita no mês
- `ultimas_transacoes` → UNION de receitas e despesas, ordenado por data DESC (10 itens)
- `proximos_vencimentos` → Despesas pendentes com vencimento nos próximos 7 dias
- `metas` → Contagem e totais de metas ativas
- `emprestimos / financiamentos` → Resumo de dívidas ativas
- `limites` → Limites do plano do usuário (para o frontend ajustar UI)

Interdependências do Dashboard

O Dashboard lê de (mas não escreve em) todos estes módulos:

`receitas` ← `despesas` ← `card_charges` ← `cartoes` ← `investimentos` ← `metas` ← `emprestimos` ← `financiamentos` ← `despesas.vencimento`

6. RECEITAS

Rota: `/api/receitas` — `src/routes/receitas.ts`

Cadastro de receita

```
POST /api/receitas {
  descricao: string (obrigatório)
  data: string (obrigatório – YYYY-MM-DD)
  categoria: string (obrigatório – ex: "Salário", "Freelance", "Investimentos")
  valor: number (obrigatório)
  recorrente?: boolean (padrão: false)
  frequencia?: string (mensal, semanal, anual – usado com recorrente=true)
  observacoes?: string
}
```

Limite FREE: 10 receitas/mês. Superado o limite → 403 com `upgrade`: `true`.

Conquista automática: Ao criar a primeira receita → `primeira_receita` (10pts).

Filtros disponíveis

```
GET /api/receitas?mes=3&ano=2026&categoria=Salário&limit=50&offset=0
```

Categorias comuns usadas

Salário, Freelance, Investimentos, Poupança, Aluguel, Dividendos, Outros

Onde as receitas aparecem em outros módulos

Módulo	Como usa
Dashboard	SUM do mês atual, gráfico de 6 meses, últimas transações
Regra 50/30/20	SUM do mês como <code>income</code> base para os cálculos
Projeção Financeira	Média dos últimos 6 meses
Comparativo Mensal	Agrupado por mês
Relatório Anual	Agrupado por mês para o ano inteiro
IA / Diagnóstico	Análise de fluxo de caixa
Simulador Amortização	Fluxo de caixa mensal para recomendação

7. DESPESAS

Rota: `/api/despesas` — `src/routes/despesas.ts`

Este é o módulo **mais complexo** do sistema, com lógica de parcelamento, integração com cartão e sincronização bidirecional.

Campos de uma despesa

```
{
  descricao: string // obrigatório
  data: string // data da compra (YYYY-MM-DD)
  categoria: string // obrigatório (Alimentação, Moradia, Transporte, etc.)
  subcategoria?: string
  valor: number // total (dividido por parcelas internamente)
  parcelado?: boolean // se true, cria N registros
  numero_parcelas?: int // quantas parcelas
  status: 'pago'|'pendente'
  fixa_ou_variavel: 'fixa'|'variavel'
  recorrente?: boolean
  vencimento?: string // data de vencimento (YYYY-MM-DD)
  observacoes?: string
  meio_pagamento: 'dinheiro'|'pix'|'debito'|'cartao_credito'|'parcelado_cartao'|'outros'
  cartao_id?: int // obrigatório se meio_pagamento = cartao_credito/parcelado_cartao
}
```

Limite FREE: 30 despesas/mês.

Lógica de parcelamento

Quando `parcelado=true` e `numero_parcelas=12`:

- O backend cria **12 registros separados** na tabela `despesas`
- Cada parcela tem: `descricao = "Nome (1/12)"`, `data` incrementando mês a mês
- O `valor` é dividido igualmente: `valor_parcela = total / 12`
- Se houver cartão vinculado, cada parcela calcula seu `billing_month/year` e cria um `card_charge` correspondente

Lógica de fatura do cartão

Quando `meio_pagamento = 'cartao_credito'` e `cartao_id` preenchido:

1. Busca dados do cartão (`dia_fechamento`, `dia_vencimento`)
2. Para cada parcela:
 - Se `dia da compra >= dia_fechamento` → vai para a PRÓXIMA fatura
 - Senão → vai para a fatura do mês atual
 - `billing_month` e `billing_year` são calculados
 - `data_vencimento = dia_vencimento` do mês da fatura
3. Cria `card_charge` em `card_charges` (fonte de verdade do cartão)
4. Deduz do `limite_disponivel` do cartão

Sincronização bidirecional (PATCH /despesas/:id/status)

Quando o status de uma despesa muda:

- Atualiza `despesas.status`
- Se a despesa tem `cartao_id` → busca o `card_charge` vinculado via `expense_id`
- Atualiza `card_charges.status` para manter sincronia
- Se pagando: restaura `limite_disponivel` do cartão
- Se despagando: decrementa `limite_disponivel` do cartão

Conquistas automáticas nas despesas

- `disciplinado` → 10 despesas pagas no mesmo mês
- `poupador` → receitas > despesas em 20%+ no mês

Onde as despesas aparecem

Módulo	Como usa
Dashboard	Critério temporal duplo (pago=data, pendente=vencimento)
Orçamentos	Gasto real vs limite por categoria
Regra 50/30/20	Classificação em Necessidades/Desejos
Comparativo	Agrupado por mês
Detector de Assinaturas	Analisa padrões de despesas pagas dos últimos 8 meses
IA / Diagnóstico	Análise de categorias e comprometimento
Relatório	Agrupado por mês
Tags	Filtro adicional por tag
Alertas Cartão	Monitora gastos por cartão

8. CARTÕES DE CRÉDITO

Rota: `/api/cartoes` — `src/routes/cartoes.ts`

Cadastro do cartão

```
POST /api/cartoes {
  nome: string // ex: "Nubank Roxinho"
  bandeira: string // Visa, Mastercard, Elo, etc.
  banco: string // Nome do banco
  limite_total: number // Limite total
  dia_vencimento: int // Dia do vencimento da fatura (ex: 10)
  dia_fechamento: int // Dia de fechamento da fatura (ex: 3)
  cor?: string // Cor hex para o card visual
  ultimos_digitos?: string // 4 últimos dígitos
}
```

Limite FREE: 2 cartões.

Cálculo de uso

O limite utilizado é calculado **dinamicamente** via `card_charges` (não o campo `limite_disponivel` da tabela, que pode estar desatualizado):

```
SELECT COALESCE(SUM(valor),0) as total
FROM card_charges
WHERE card_id = ? AND status = 'pendente'
```

Resultado: `limite_utilizado`, `limite_disponivel`, `percentual_uso`.

Gestão de faturas

```
GET /api/cartoes/:id/fatura?mes=3&ano=2026
→ Retorna todos os card_charges do billing_month/year informado
→ Subtotal de compras, subtotal de parcelas, total da fatura

POST /api/cartoes/:id/pagar-fatura { mes, ano }
→ Marca todos os card_charges do período como 'pago'
→ Atualiza despesas vinculadas para 'pago'
→ Restaura limite_disponivel do cartão
→ Conquista: 'fatura_paga'

POST /api/cartoes/:id/lancamento { descricao, valor, data, categoria, parcelas }
→ Cria despesa + card_charge com cálculo bancário correto
→ Decrementa limite_disponivel
```

Cálculo da data de vencimento da fatura

```
// Compra em 20/jan, fechamento dia 3:
// 20 >= 3 → vai para fatura de fevereiro
// Vencimento: dia X de fevereiro

function calcBillingPeriod(purchaseDate, closingDay) {
  if (purchaseDate.getDate() >= closingDay) {
    month++ // próxima fatura
  }
  return { month, year }
}
```

Alertas de Cartão

Rota: GET /api/alertas-cartao — analisa todos os cartões do usuário e retorna:

- Cartões com uso > 80% do limite
- Faturas com vencimento nos próximos 5 dias
- Compras suspeitas (valor alto único)

9. METAS FINANCEIRAS

Rota: /api/metask — src/routes/metask.ts

Tipos de meta

Categoria	Conquista automática
imovel / nome inclui "casa"	meta_casa
veiculo / nome inclui "carro"	meta_carro
viagem / nome inclui "viagem"	meta_viagem
educacao / nome inclui "curso"	meta_educacao
liberdade / nome inclui "fire"	meta_liberdade
aposentadoria	meta_aposentadoria
Qualquer meta	planejador (25pts)

Campos de uma meta

```
nome, descricao, valor_objetivo, valor_atual (progresso atual),
data_meta (prazo), categoria, cor (#hex), icone (string),
linked_debt_type, linked_debt_id (para metas de quitação de dívidas)
```

Limite FREE: 3 metas.

Meta de Quitar Dívidas (debt_payoff)

Categoria especial que vincula a meta a um financiamento ou empréstimo:

```
POST /api/metast {
  categoria: 'debt_payoff',
  linked_debt_type: 'all' | 'financiamento' | 'emprestimo' | 'especifico',
  linked_debt_id?: int // para 'especifico'
}
```

O backend calcula automaticamente:

- `valor_objetivo` = `saldo_devedor_total` + `valor_já_pago` (dívida original)
- `valor_atual` = `valor_já_pago` (progresso)

Sincronização automática de dívidas

```
POST /api/metast/sincronizar-dividas
→ Para todas as metas debt_payoff ativas
→ Recalcula valor_atual com base no saldo devedor atual
→ Se saldo = 0, marca meta como 'concluida'
→ Conquista 'sem_dividas' se quitou tudo
```

Depósito manual em meta

```
PATCH /api/metast/:id/deposito { valor }
→ Incrementa valor_atual
→ Se valor_atual >= valor_objetivo → status = 'concluida'
→ Retorna { novo_valor, status, message }
```

Métricas calculadas por meta

Para cada meta retornada pelo GET:

- `percentual` = $(\text{valor_atual} / \text{valor_objetivo}) \times 100$
- `meses_restantes` = `dias_restantes` / 30
- `valor_faltante` = `valor_objetivo` - `valor_atual`
- `mensalidade_necessaria` = `valor_faltante` / `meses_restantes`

Interdependência com outros módulos

- **Dashboard** → conta metas ativas, exibe objetivo vs atual
- **IA / Diagnóstico** → analisa progresso das metas para o score
- **Empréstimos/Financiamentos** → metas `debt_payoff` sincronizam com `saldo_devedor`

10. ORÇAMENTOS

Rota: `/api/orcamentos` — `src/routes/orcamentos.ts`

Conceito

Orçamentos definem um **teto mensal por categoria**. O sistema compara o limite definido com o gasto real das despesas naquela categoria.

Cadastro

```
POST /api/orcamentos {
  categoria: string // uma das 15 categorias disponíveis
  limite: number // valor máximo para o mês
  mes: int // mês (1-12)
  ano: int // ano
  alerta_percentual: int // alerta quando atingir X% (padrão: 80)
}
```

Categorias disponíveis:

alimentacao, moradia, transporte, saude, educacao, lazer, vestuario, beleza, pets, assinaturas, tecnologia, viagem, outros, fixo, supermercado

Como o gasto real é calculado

Para cada orçamento do período, o sistema busca:

```
SELECT SUM(valor) FROM despesas
WHERE user_id = ? AND categoria = ?
AND strftime('%m', COALESCE(vencimento, data)) = ? -- mês do orçamento
AND strftime('%Y', COALESCE(vencimento, data)) = ? -- ano do orçamento
AND status IN ('pago', 'pendente') -- considera tudo
```

Status do orçamento

Percentual gasto vs limite	Status
> 100%	exceeded (vermelho)
≥ alerta_percentual (padrão 80%)	warning (laranja)
≥ 70%	attention (amarelo)
< 70%	ok (verde)

Sugestão de categorias sem orçamento

O endpoint retorna também `semOrçamento` — lista das categorias que ainda não têm orçamento configurado para o período, para incentivar o usuário a completar o planejamento.

Interdependências

- **Despesas** → fonte do gasto real (critério de data = vencimento ou data)
- **Regra 50/30/20** → usa categorias semelhantes mas com lógica própria
- **IA** → analisa se os orçamentos foram respeitados

11. RECORRÊNCIAS

Rota: `/api/recorrencias` — `src/routes/recorrencias.ts`

Requer: Plano Premium ou Pro (FREE → erro 403)

Conceito

Transações que se repetem todo mês. Ao configurar uma recorrência, o usuário define uma **regra** que pode ser executada para gerar despesas ou receitas no mês corrente.

Cadastro

```
POST /api/recorrencias {
  tipo: 'despesa' | 'receita' (obrigatório)
  descricao: string (obrigatório)
  valor: number (obrigatório)
  categoria: string (obrigatório)
  dia_vencimento: int (obrigatório – dia do mês)
  meio_pagamento?: string
  data_fim?: string (YYYY-MM-DD – quando parar)
}
```

Geração da transação do mês

```
POST /api/recorrencias/:id/gerar
→ Cria despesa ou receita usando os dados da recorrência
→ data = dia_vencimento do mês atual
→ Atualiza ultimo_gerado na recorrência
→ Retorna { success, message, lancamento_id }
```

A UI exibe botão "Gerar Agora" para cada recorrência que ainda não foi gerada no mês.

Campo `gerada_mes_atual`

Calculado em runtime: se `ultimo_gerado >= primeiro dia do mês atual` → true.

Conquistas

- `automatico` → Primeira recorrência criada (25pts)
- Conquista por 5+ recorrências ativas

Interdependência

- Gera **Receitas** e **Despesas** reais no banco
- O Dashboard passa a incluir esses lançamentos

12. LEMBRETES

Rota: `/api/lembretes` — `src/routes/lembretes.ts`

Diferença entre Lembrete e Recorrência

Plano	FREE (até 5)	Premium+
Ação	Alerta visual no sidebar	Gera lançamento automático

Plano	FREE (até 5)	Premium+
Propósito	Lembrar de pagar	Pagar automaticamente

Cadastro

```
POST /api/lembretes {
  titulo: string (obrigatório)
  descricao?: string
  tipo: 'conta'|'boleto'|'cartao'|'imposto'|'outros'
  valor_estimado?: number
  dia_vencimento?: int (dia do mês)
  frequencia: 'mensal'|'bimestral'|'trimestral'|'semestral'|'anual'
  alertar_dias_antes: int (padrão: 3)
}
```

Limite FREE: 5 lembretes ativos.

Cálculo de urgência

Para cada lembrete com `dia_vencimento`:

- Calcula a data de vencimento do mês atual
- Se já passou → projeta para o próximo mês
- `diasParaVencer` = diferença em dias
- `urgente` = `diasParaVencer` $\leq$ `alertar_dias_antes`

O badge de urgência no sidebar é calculado pela contagem de lembretes urgentes com `status_mes` = `'aguardando'`.

Ações

```
PATCH /api/lembretes/:id/status { status: 'pago'|'ignorado'|'aguardando' }
→ Atualiza status do mês atual
→ Registra em lembretes_historico com data e valor real pago
```

Conquista

- `lembrete_mestre` → 5 lembretes cadastrados (20pts)

13. INVESTIMENTOS E CAIXINHA CDI

Rota: `/api/investimentos` — `src/routes/investimentos.ts`

Tipos de investimento suportados

Tipo	Rentabilidade
caixinha	% do CDI, capitalização diária em dias úteis (252/ano)
tesouro_direto	Taxa anual informada

Tipo	Rentabilidade
cdb	Taxa anual informada
lci / lca	Taxa anual informada
acoes	Taxa anual informada
fii	Taxa anual informada
cripto	Taxa anual informada
poupanca	Taxa anual informada
outros	Taxa anual informada

Limite FREE: 3 investimentos.

Caixinha CDI — Lógica Especial

```
// Fórmula de capitalização diária:
const cdiDiario = (1 + CDI_anual/100) ^ (1/252) - 1
const taxaDiaria = cdiDiario * (percentual_cdi / 100)
const valorAtual = valorInvestido * (1 + taxaDiaria) ^ diasDecorridos
```

Campos adicionais para Caixinha:

- `percentual_cdi` → porcentagem do CDI (ex: 100 = 100% do CDI)
- `cdi_atual` → taxa CDI anual vigente (buscada da tabela `cdi_historico`)
- `data_ultimo_calculo` → data do último cálculo de rendimento

O valor atual da Caixinha é **recalculado a cada GET**, não armazenado estático.

Conquistas de Investimento

Conquista	Gatilho
<code>investidor</code>	Primeiro investimento de qualquer tipo
<code>investidor_cdi</code>	Tipo = caixinha
<code>investidor_acoes</code>	Tipo = ações
<code>investidor_fii</code>	Tipo = fii
<code>investidor_cripto</code>	Tipo = cripto
<code>investidor_tesouro</code>	Tipo = tesouro_direto
<code>investidor_cdb</code>	Tipo = cdb
<code>poupador_dedicado</code>	Total atual ≥ R\$ 10.000
<code>milionario</code>	Total atual ≥ R\$ 100.000
<code>investidor_diversificado</code>	3+ tipos diferentes

Interdependências

- **Dashboard** → exibe `total_investido`, `total_atual`, rendimento
- **IA / Diagnóstico** → analisa se os investimentos são suficientes vs renda mensal
- **CDI Real** → atualiza `cdi_atual` via GET `/api/cdi`
- **Simulador de Investimentos** → simula cenários sem criar registros
- **Regra 50/30/20** → depósitos em investimentos somam ao grupo "Poupança"

14. RESERVA DE EMERGÊNCIA (LEGADO)

Rota: `/api/reserva` — `src/routes/reserva.ts`

Nota: Esta é a reserva legada (uma por usuário). O sistema v3.0 introduziu as Reservas Especializadas (múltiplas). Ambas coexistem.

Conceito

Reserva de emergência única, com objetivo em meses de despesas cobertas.

Dados calculados automaticamente

```
GET /api/reserva
→ Calcula média de despesas dos últimos 3 meses
→ ideal = média × objetivo_meses
→ percentual_cobertura = (valor_atual / ideal) × 100
→ meses_cobertos = valor_atual / média_despesas
```

Conquistas

- `reserva_1_mes` → reserva cobre 1+ mês
- `reserva_3_meses` → cobre 3+ meses (60pts, épico)
- `reserva_6_meses` → cobre 6+ meses (80pts, épico)
- `reserva_completa` → atingiu 100% da meta (100pts, lendário)

15. MÚLTIPLAS RESERVAS ESPECIALIZADAS (v3.0)

Rota: `/api/reservas-esp` — `src/routes/reservas-especializadas.ts`

Tipos de reserva disponíveis

Tipo	Ícone	Cor	Meses sugeridos	Prioridade
emergency	■	Vermelho	6	1 (maior)
unemployment	■	Âmbar	12	1
health	■	Azul	2	2
vehicle	■	Verde-lima	12	2
family	■	Laranja	3	2

Tipo	Ícone	Cor	Meses sugeridos	Prioridade
education	■	Ciano	6	3
travel	✈️ ■	Roxo	12	4
event	■	Rosa	8	4
custom	■	Índigo	6	5 (menor)

Limites por plano

Plano	Máximo de reservas simultâneas
FREE	1
PREMIUM	3
PRO	Ilimitado

Fluxo completo

1. Criar reserva

```
POST /api/reservas-esp {
  type: 'emergency' // tipo obrigatório
  name: string // nome personalizado
  description?: string
  target_amount: number // meta em R$
  current_amount?: number // saldo inicial (padrão: 0)
  deadline?: string // prazo (YYYY-MM-DD)
  monthly_target?: number // aporte mensal sugerido
  priority?: int // 1-5 (padrão: do tipo)
}
```

Se `current_amount > 0` → cria automaticamente uma `reserve_transaction` tipo `'deposit'` com descrição "Saldo inicial".

2. Depositar

```
POST /api/reservas-esp/:id/depositar { amount, description? }
→ Valida: reserva ativa e do usuário
→ Limita ao target_amount (não passa da meta)
→ Registra em reserve_transactions (tipo = 'deposit')
→ Atualiza current_amount na specialized_reserves
→ Se current_amount >= target_amount → status = 'completed', conquista 'reserva_spec_completa'
→ Retorna { new_amount, percent_complete, completed, message }
```

3. Sacar

```
POST /api/reservas-esp/:id/sacar { amount, description? }
→ Valida: amount <= current_amount
→ Registra em reserve_transactions (tipo = 'withdrawal')
→ Status volta para 'active' (mesmo se estava 'completed')
→ Retorna { new_amount, message }
```

4. Histórico

```
GET /api/reservas-esp/:id/historico
→ Retorna últimas 50 transações (depósitos e saques) desta reserva
```

5. Resumo consolidado (GET /)

Retorna todas as reservas ativas + summary:

- `total_saved` → soma de todos os `current_amount`
- `total_target` → soma de todos os `target_amount`
- `overall_progress` → percentual geral
- `active_count` / `completed_count`

Conquistas

Conquista	Gatilho
<code>multi_reserva_criada</code>	Criar qualquer reserva especializada (30pts)
<code>multi_3_reservas</code>	Ter 3 reservas ativas simultâneas (80pts, épico)
<code>reserva_spec_completa</code>	Completar uma reserva especializada (100pts, lendário)

Relação com Regra 50/30/20

Depósitos em reservas especializadas somam ao grupo **"Poupança"** na análise 50/30/20.

16. FINANCIAMENTOS

Rota: `/api/financiamentos` — `src/routes/financiamentos.ts`

Conceito

Financiamentos imobiliários ou de veículos com parcelas longas (até 360 meses).

Campos obrigatórios

```
descricao, valor_imovel, valor_financiado, taxa_juros_anual,
numero_parcelas, valor_parcela, data_inicio
```

Campos opcionais

```

tipo_imovel: 'residencial'|'comercial'|'rural'|'terreno'
tipo_bem: 'imovel'|'veiculo'
valor_entrada, parcelas_pagas, banco, contrato
sistema_amortizacao: 'price'|'sac'|'sacre'
indexador: 'prefixado'|'ipca'|'tr'|'cdi'
observacoes

```

Limite FREE: 1 financiamento ativo.

Cálculo automático ao cadastrar

O sistema calcula o `saldo_devedor` atual usando a fórmula PRICE:

```

function calcularSaldoDevedor(valorFinanciado, taxaMensal, totalParcelas, parcelasPagas) {
  // Saldo após parcelasPagas parcelas pelo sistema PRICE
  const f = (1 + taxaMensal) ^ totalParcelas
  const prestacao = valorFinanciado * (taxaMensal * f) / (f - 1)
  let saldo = valorFinanciado
  for (let i = 0; i < parcelasPagas; i++) {
    const juros = saldo * taxaMensal
    const amortizacao = prestacao - juros
    saldo -= amortizacao
  }
  return Math.max(0, saldo)
}

```

Geração automática de despesas

Ao criar um financiamento, o backend cria **automaticamente** as despesas futuras (parcelas não pagas) na tabela `despesas`:

```

Para i = parcelas_pagas até numero_parcelas:
  Cria despesa: categoria='Financiamento', status='pendente',
  data = data_inicio + i meses,
  observacoes = "Financiamento: {descricao} ({i+1}/{total})"

```

Métricas calculadas por GET

- `perc_pago` → $(\text{parcelas_pagas} / \text{total}) \times 100$
- `parcelas_restantes` → $\text{total} - \text{pagas}$
- `total_pago` → $\text{valor_parcela} \times \text{parcelas_pagas}$

Ação de pagamento de parcela

```

POST /api/financiamentos/:id/pagar { parcelas_a_pagar: int }
→ Atualiza parcelas_pagas += parcelas_a_pagar
→ Recalcula saldo_devedor
→ Verifica conquistas de percentual quitado (10%, 15%)
→ Se concluído → status = 'quitado'

```

Conquistas

Conquista	Gatilho
<code>primeiro_imovel</code>	Cadastrar primeiro financiamento imobiliário
<code>quitou_10pct</code>	Quitou 10% do valor financiado
<code>quitou_15pct</code>	Quitou 15% do valor financiado

Interdependência com outros módulos

- **Dashboard** → exibe `total_saldo_devedor` e `total_parcela_mensal` dos financiamentos ativos
- **Metas debt_payoff** → sincroniza `saldo_devedor` como progresso da meta
- **Simulador de Amortização** → usa `saldo_devedor`, `valor_parcela`, `parcelas_restantes`, `taxa_juros_anual`
- **IA** → analisa comprometimento de renda com financiamentos

17. EMPRÉSTIMOS

Rota: `/api/emprestimos` — `src/routes/emprestimos.ts`

Tipos

`pessoal`, `consignado`, `veiculo`, `estudantil`, `imobiliario`, `cheque_especial`, `cartao_credito`, `outros`

Diferença entre Financiamento e Empréstimo

Prazo típico	10–30 anos	1–5 anos
Bem vinculado	Imóvel ou veículo	Sem bem vinculado
Taxa típica	8–12% a.a.	1–10% a.m.
Campos especiais	<code>tipo_imovel</code> , <code>indexador</code>	<code>tipo</code> , <code>credor</code> , <code>dia_vencimento</code>
Taxa de entrada	<code>valor_entrada</code>	N/A

Limite FREE: 2 empréstimos ativos.

Campos obrigatórios

```
descricao, valor_original, taxa_juros_mensal, numero_parcelas, valor_parcela, data_inicio
```

Cálculo do saldo devedor

Se o usuário informar `saldo_devedor_atual` → usa esse valor diretamente.

Senão → calcula pela fórmula PRICE:

```
function calcSaldo(valorOriginal, taxaMensal, totalParcelas, parcelasPagas) {
  const prestacao = valorOriginal * (taxaMensal * (1+taxaMensal)^total) /
    ((1+taxaMensal)^total - 1)
  // Amortiza parcelasPagas meses
  let saldo = valorOriginal
  for (let i = 0; i < parcelasPagas; i++) {
    saldo = saldo * (1 + taxaMensal) - prestacao
  }
  return Math.max(0, saldo)
}
```

Geração de despesas automáticas

Igual ao Financiamento — cria despesas futuras para cada parcela não paga.

Métricas calculadas

- `perc_pago` → $(\text{parcelas_pagas} / \text{total}) \times 100$
- `total_a_pagar` → $\text{valor_parcela} \times \text{total}$
- `total_juros` → $\text{total_a_pagar} - \text{valor_original}$
- `custo_efetivo_total` → $(\text{total_juros} / \text{valor_original}) \times 100$

Conquistas

Conquista	Gatilho
primeiro_carro	Tipo = veículo
sem_dividas	Quitou o empréstimo (conquista via metas/sincronizar-dividas)

18. SIMULADOR DE AMORTIZAÇÃO INTELIGENTE (v3.0)

Rota: `/api/amortizacao` — `src/routes/amortizacao.ts`

Requer: Plano Premium ou Pro.

Conceito

Simula o que acontece quando o usuário faz um **pagamento extraordinário** (amortização) no financiamento ou empréstimo. Compara dois cenários e recomenda o melhor.

Cenário A — Reduzir Parcela

Mantém o prazo, diminui a parcela mensal:


```
// Sistema PRICE:
newInstallment = priceInstallment(newBalance, monthlyRate, remainingMonths)
// Onde newBalance = balance - extra (amortização)

// Sistema SAC:
newConstAmort = newBalance / remainingMonths
newInstallment = newConstAmort + (newBalance × monthlyRate)
```

Cenário B — Reduzir Prazo

Mantém a parcela, diminui o número de meses:

```
// Sistema PRICE:
newMonths = ceil(-log(1 - (newBalance × rate)/installment) / log(1 + rate))

// Sistema SAC:
constAmort = balance / remainingMonths
newMonths = ceil(newBalance / constAmort)
```

Recomendação Inteligente

O sistema analisa o **fluxo de caixa mensal do usuário** para fazer a recomendação:

```
const monthlyFlow = receitas_do_mês - despesas_pagas_do_mês

if (monthlyFlow < 0)
→ Recomenda: Reduzir Parcela (alívio imediato)
else if (economiaB > economiaA × 1.3)
→ Recomenda: Reduzir Prazo (mais juros economizados)
else if (reducaoParcelaA > R$200)
→ Recomenda: Reduzir Parcela (liberdade de caixa)
else
→ Recomenda: Reduzir Prazo (liberdade da dívida mais rápido)
```

Duas formas de simular

1. Com financiamento cadastrado:

```
POST /api/amortizacao/simular {
  financing_id: int,
  amortization_amount: number
}
```

O backend busca automaticamente `saldo_devedor`, `valor_parcela`, `parcelas_restantes`, `taxa_juros_anual`, `sistema_amortizacao` do financiamento.

2. Entrada manual:

```
POST /api/amortizacao/simular {
  manual_balance: number,
  manual_installment: number,
  manual_remaining_months: int,
  manual_annual_rate: number,
  manual_system: 'PRICE' | 'SAC',
  amortization_amount: number
}
```

Histórico de simulações

```
GET /api/amortizacao/historico
→ Últimas 20 simulações, com nome do financiamento vinculado
→ Inclui todos os campos calculados (nova parcela, juros economizados, etc.)
```

Conquista

- **amortizou_simulou** → Usou o simulador de amortização (30pts, raro)

Interdependências

- **Financiamentos** → fonte dos dados quando **financing_id** é informado
- **Receitas + Despesas** → usados para calcular o fluxo de caixa da recomendação
- **Dashboard** → o **comprometimento_dividas_pct** orienta quando vale amortizar

19. DETECTOR DE ASSINATURAS FANTASMA (v3.0)

Rota: `/api/assinaturas-fantasma` — `src/routes/assinaturas-fantasma.ts`

Como funciona o algoritmo

Passo 1 — Coleta de dados

```
SELECT id, descricao, valor, data, categoria FROM despesas
WHERE user_id = ?
AND data >= date('now', '-8 months')
AND status = 'pago'
AND valor >= 5.0 -- ignora valores muito pequenos
ORDER BY data ASC
```

Mínimo de 6 despesas pagas para iniciar a análise. Se insuficiente → retorna `insufficient_data: true`.

Passo 2 — Agrupamento

Despesas são agrupadas por chave `{descricao_normalizada}|{valor_arredondado}`:

```
// Normalização da descrição:
desc.toLowerCase()
.replace(/^[a-z0-9\s]/g, ' ') // remove caracteres especiais
.replace(/\s+/g, ' ')
.trim()
.substring(0, 30)

// Valor arredondado: Math.round(valor × 50) / 50
// Agrupa variações de centavos (±2%)
```

Passo 3 — Análise de periodicidade

Para grupos com ≥ 3 ocorrências, calcula:

- `avgInterval` → média de dias entre ocorrências
- `stdDev` → desvio padrão (mede regularidade)

Padrões detectados:

Padrão	Intervalo	Desvio máximo
Mensal	25–37 dias	< 8 dias
Quinzenal	12–18 dias	< 4 dias
Anual	335–395 dias	Qualquer (mín 2 ocorrências)

Passo 4 — Cálculo de confiança

Critério	Pontos adicionados
Keyword de serviço conhecido	+40
Padrão mensal ou quinzenal	+30
≥ 6 ocorrências	+20
<code>stdDev</code> < 3 (muito regular)	+10
Padrão anual	Mínimo 65

Só é salvo se `confidence` ≥ 60 .

Passo 5 — Serviços conhecidos (keyword matching)

20+ categorias detectadas automaticamente:

- Streaming: Netflix, Spotify, Amazon Prime, Disney+, HBO/Max, Globoplay, YouTube Premium, Apple Music, Deezer
- Cloud: iCloud, Dropbox, OneDrive/Office365, Google One
- Software: Adobe, Canva, ChatGPT/OpenAI, Claude, Copilot
- Fitness: SmartFit, BodyTech, academia, Bluefit
- Transporte: Uber One
- Alimentação: iFood, Rappi Prime
- Gaming: Xbox Game Pass, PlayStation Plus, Nintendo

- Profissional: LinkedIn Premium
- Educação: Duolingo Plus

Passo 6 — Upsert no banco

Para cada assinatura detectada:

- Se já existe (mesmo `normalized_description + valor`) → atualiza frequência e última ocorrência
- Se nova → insere na `detected_subscriptions`
- Status inicial: `'detected'`

Feedback do usuário

```
PATCH /api/assinaturas-fantasma/:id/feedback { feedback: 'use_regularly' | 'want_cancel' | 'ignore' }
```

Feedback	Ação	Status final
use_regularly	Confirma uso regular	confirmed
want_cancel	Marca para cancelar	cancelled
ignore	Ignora esta detecção	ignored

O GET principal retorna apenas status `detected` e `confirmed`.

Conquistas

Conquista	Gatilho
sub_detector_scanned	Escaneou e encontrou algo (20pts)
sub_cancelou_1	Marcou uma assinatura para cancelar (40pts, raro)

20. REGRA 50/30/20 AUTOMATIZADA (v3.0)

Rota: GET `/api/regra-503020?mes=M&ano=A` — `src/routes/regra-503020.ts`

A regra

- **50%** da renda → Necessidades (moradia, alimentação, saúde, transporte)
- **30%** da renda → Desejos (lazer, assinaturas, delivery, viagem)
- **20%** da renda → Poupança (investimentos, reservas)

Como as despesas são classificadas

Necessidades (50%):

Alimentação, Moradia, Saúde, Transporte, Educação, Contas, Mercado, Farmácia

Desejos (30%):

Lazer, Viagem, Roupas, Assinaturas, Delivery, Restaurante, Beleza, Entretenimento, Pets, Eletrônicos, Outros

Poupança (20%):

Investimentos, Poupança (categorias de despesa)

+ Investimentos cadastrados no período

+ Depósitos em reservas especializadas no período

Score de aderência (0-100)

```
// Pesos: Necessidades (30%), Desejos (30%), Poupança (40%)
needsScore = max(0, 100 - |percentNeeds - 50| × 2)
wantsScore = max(0, 100 - |percentWants - 30| × 3)
savingsScore = max(0, min(100, percentSavings × 5))

score = round(needsScore × 0.3 + wantsScore × 0.3 + savingsScore × 0.4)
```

Recomendações automáticas

Situação	Recomendação gerada
Necessidades > R\$300 acima do ideal	"■ Necessidades estão R\$X acima do ideal..."
Desejos > R\$200 acima do ideal	"■ Lazer/Desejos estão R\$X acima do ideal..."
Poupança abaixo do ideal	"■ Poupança abaixo do ideal. Tente guardar mais R\$X/mês"
Poupança ≥ 20%	"■ Você poupa X% da renda..."
Score ≥ 80	"■■ Excelente equilíbrio financeiro!"

Conquista

- `regra_503020_verde` → Score ≥ 80 (60pts, épico)

Interdependências

Módulo	Como usa
Receitas	SUM do mês como <code>income</code> base
Despesas	Agrupadas por categoria para classificação
Investimentos	SUM de <code>valor_investido</code> do mês vai para Poupança
Reservas Especializadas	SUM de depósitos do mês vai para Poupança

21. DESAFIO 52 SEMANAS (v3.0)

Rota: `/api/desafio-52` — `src/routes/desafio-52.ts`

Como funciona

Na semana 1, guarda R\$ 1. Na semana 2, R\$ 2. ... Na semana 52, R\$ 52.

Total acumulado: R\$ 1.378 no ano.

Inicialização automática

No primeiro acesso do ano, o backend cria automaticamente todos os 52 registros:

```
for (let w = 1; w <= 52; w++) {  
  INSERT INTO weekly_challenges (user_id, year, week_number, target_amount, status)  
  VALUES (userId, year, w, w, 'pending') // semana N = R$ N  
}
```

E concede a conquista `desafio_52_iniciou`.

Status de cada semana

Status	Significado
pending	Ainda não guardou
completed	Guardou o valor da semana
skipped	Pulou esta semana

Ação de marcação

```
PATCH /api/desafio-52/:semana?ano=2026 { status: 'completed' | 'skipped' | 'pending' }  
→ Atualiza o status da semana específica  
→ Se completed → completed_at = datetime('now')  
→ Retorna { message } com valor guardado ou incentivo
```

O toggle na UI faz ciclo: pending → completed → skipped → pending.

Resumo retornado

```
{  
  "summary": {  
    "completed": 15, // semanas completadas  
    "pending": 35, // semanas pendentes  
    "skipped": 2, // semanas puladas  
    "total_saved": 120.00, // valor já guardado (soma das semanas completadas)  
    "total_target": 1378.00, // meta total do ano  
    "progress_pct": 28 // percentual de semanas completas  
  },  
  "current_week": 12 // semana atual do ano  
}
```

Conquistas

Conquista	Gatilho
desafio_52_iniciou	Primeiro acesso ao desafio (25pts)
desafio_52_metade	26+ semanas completadas (75pts, épico)

Conquista	Gatilho
desafio_52_completo	52 semanas completadas (200pts, lendário)

22. PROJEÇÃO FINANCEIRA

Rota: GET /api/projecao — src/routes/projecao.ts

Requer: Plano Premium ou Pro.

Como funciona

- **Coleta** receitas e despesas dos últimos 6 meses reais
- **Calcula tendência** por regressão linear simples (slope da reta)
- **Média ponderada** com pesos crescentes (meses mais recentes têm mais peso: [1, 1, 1.5, 2, 2.5, 3])
- **Calcula confiança** baseada no coeficiente de variação ($1 - |CV|$)
- **Projeta** os próximos 3 e 6 meses usando:
 - Tendência linear + sazonalidade histórica
 - Saldo acumulado projetado

Indicadores de tendência

Slope	Tendência
> +R\$50	positive → crescendo
< -R\$50	negative → caindo
Entre -50 e +50	stable → estável

Retorno

```
{
  "historico": [...], // 6 meses reais
  "projecao_3m": [...], // próximos 3 meses
  "projecao_6m": [...], // próximos 6 meses
  "tendencia": "positive",
  "confianca": 75, // % de confiança na projeção
  "slope": 120.50, // variação mensal estimada
  "media_saldo": 850.00
}
```

23. COMPARATIVO MENSAL

Rota: GET /api/comparativo — src/routes/comparativo.ts

Funcionalidade

Compara receitas e despesas entre dois meses diferentes, mostrando variações em R\$ e percentual.

```
GET /api/comparativo?mes_atual=3&ano_atual=2026&mes_anterior=2&ano_anterior=2026
```

Retorno

- Receitas e despesas dos dois períodos
- Variação em R\$ e %
- Top 5 categorias de cada período
- Análise: quais categorias cresceram, quais reduziram

24. RELATÓRIOS

Rota: GET /api/dashboard/relatorio?ano=2026 — src/routes/dashboard.ts

Requer: Plano Premium ou Pro.

Retorno

- Receitas e despesas para cada um dos 12 meses do ano
- Totais anuais
- Resumo de dívidas ativas (empréstimos + financiamentos)
- Resumo de metas (ativas, concluídas, objetivos)
- Resumo de investimentos (total investido, valor atual)

A UI gera um gráfico de barras interativo com Chart.js e permite exportação em PDF.

Conquista

- **analista** → Consultou o relatório (30pts, raro)

25. SIMULADOR DE INVESTIMENTOS

Rota: GET /api/investimentos/simulacao — src/routes/investimentos.ts

Requer: Plano Premium ou Pro.

Parâmetros

```
?valor=10000&tipo=caixinha&prazo_meses=24&percentual_cdi=110
```

Taxas mensais padrão por tipo

Tipo	Taxa Mensal Estimada
Poupança	0.5%
CDB	0.9%
LCI/LCA	0.85%

Tipo	Taxa Mensal Estimada
Tesouro Direto	0.83%
Ações	1.2%
FII	0.8%
Cripto	2.0%
Caixinha CDI	Calculada dinamicamente
Outros	0.7%

Para **caixinha**, usa o CDI real da tabela **cdi_historico** e calcula:

```
CDI mensal = (1 + CDI_aa/100)^(1/12) - 1
Taxa caixinha = CDI mensal × (percentual_cdi / 100)
```

Retorno

- **valor_final**, **lucro_total**, **rentabilidade_total**
- **projecao** → pontos trimestrais até o prazo final
- **aviso** → texto sobre garantia dos dados

26. ASSISTENTE IA / DIAGNÓSTICO 360°

Rota: GET /api/ia/insights — src/routes/ia.ts

Requer: Plano Premium ou Pro.

Conceito

Não usa uma API de LLM externa. É uma **análise 100% determinística** baseada em 11 módulos de dados do usuário, com scores e recomendações calculadas por fórmulas.

Os 11 módulos analisados

#	Módulo	Fonte	Score 0-100
1	Fluxo de Caixa	receitas, despesas	scoreCashFlow(saldo, receita)
2	Reserva de Emergência	reserva_emergencia	scoreEmergency(meses_cobertos)
3	Controle de Dívidas	emprestimos, financiamentos	scoreDebt(ratio, taxaMaxima)
4	Investimentos	investimentos	scoreInvestments(total, receita)
5	Metas	metas	scoreGoals(count, percAtingido)

#	Módulo	Fonte	Score 0-100
6	Orçamentos	orcamentos	Percentual de orçamentos respeitados
7	Cartões	card_charges	Uso médio dos cartões
8	Despesas por categoria	despesas	Top categorias vs renda
9	Lembretes	lembretes	Lembretes urgentes em aberto
10	Receitas	receitas	Diversidade de fontes
11	Evolução	histórico 6 meses	Tendência de melhora

Score global e veredicto

```
const scoreGlobal = média_ponderada_dos_11_scores

// Veredictos:
// ≥ 85 → "■ Saúde Financeira Excelente"
// ≥ 70 → "■ Finanças Bem Organizadas"
// ≥ 55 → "■ Momento de Construção"
// ≥ 35 → "■ Atenção Necessária"
// < 35 → "■ Situação Crítica – Ação Imediata"
```

Retorno

Para cada módulo:

- **score** (0-100)
- **status** (EXCELENTE | BOM | ATENCAO | CRITICO)
- **resumo** → texto explicativo
- **recomendacoes** → lista de ações concretas
- **alertas** → problemas urgentes detectados
- **oportunidades** → melhorias identificadas

Diagnóstico 360° (GET /api/ia/diagnostico-360)

Endpoint separado que retorna uma visão mais aprofundada com:

- Análise de perfil de investidor
- Comparativo com benchmarks do plano da vida financeira
- Plano de ação prioritário (3–5 ações mais impactantes)

27. TAGS E FILTROS

Rota: /api/tags — src/routes/tags.ts

Conceito

Tags são etiquetas customizadas que o usuário cria e aplica em despesas para filtros avançados além da categoria.

Criar tag

```
POST /api/tags { nome, cor, descricao? }
```

Aplicar tag a despesa

```
POST /api/despesas/:id/tags { tag_id: int }  
→ Cria relação em despesa_tags
```

Filtrar despesas por tag

```
GET /api/despesas?tag_id=3  
→ JOIN com despesa_tags para filtrar
```

Conquista

- **tagger** → Criou e usou tags em 5 despesas (25pts)

28. ALERTAS DE CARTÃO

Rota: `GET /api/alertas-cartao` — `src/routes/alertas-cartao.ts`

Tipos de alertas gerados automaticamente

- **Uso elevado** → Cartão com > 80% do limite utilizado
- **Fatura próxima** → Vencimento nos próximos 5 dias
- **Compra incomum** → Valor único acima de X vezes a média das compras

Configurar alertas manuais

```
POST /api/alertas-cartao {  
  cartao_id: int  
  tipo: 'limite_percentual' | 'valor_unico' | 'vencimento_dias'  
  valor_referencia: number // % para limite, R$ para valor, dias para vencimento  
  ativo: boolean  
}
```

CDI em Tempo Real

Rota: `GET /api/cdi` — `src/routes/cdi.ts`

Busca a taxa CDI atual do Banco Central do Brasil:

- Tenta buscar da tabela `cdi_historico` (cache)
- Se desatualizado ou vazio → faz chamada à API pública do BCB

- Salva na tabela `cdi_historico` com a data
- Retorna `{ taxa_diaria, taxa_mensal, taxa_anual, data_referencia }`

A taxa CDI é usada por:

- Caixinha CDI → cálculo de rendimento diário
- Simulador de Investimentos → taxa para tipo=caixinha

29. CONQUISTAS E GAMIFICAÇÃO

Rota: `/api/conquistas` — `src/routes/conquistas.ts`

Catálogo completo de conquistas

Grupo: Primeiros Passos

Código	Título	Como desbloquear	Pontos	Raridade
<code>primeira_receita</code>	Primeira Receita	Registrar a 1ª receita	10	Comum
<code>organizador</code>	Organizador	Registrar a 1ª despesa	10	Comum
<code>investidor</code>	Investidor	Cadastrar qualquer investimento	20	Comum
<code>sonhador</code>	Sonhador	Criar a 1ª meta	15	Comum
<code>planejador</code>	Planejador	Criar uma meta e completar perfil	25	Raro
<code>carteirinha</code>	Carteirinha	Cadastrar o 1º cartão	15	Comum

Grupo: Investimentos

Código	Título	Como desbloquear	Pontos	Raridade
<code>investidor_cdi</code>	Caixinha CDI	Investimento tipo caixinha	20	Comum
<code>investidor_acoes</code>	Investidor da Bolsa	Investimento em ações	30	Raro
<code>investidor_fii</code>	Renda Passiva Imob.	Investimento em FII	30	Raro
<code>investidor_cripto</code>	Crypto Holder	Investimento em cripto	25	Raro
<code>investidor_tesouro</code>	Tesouro Direto	Investimento em T. Direto	20	Comum
<code>investidor_cdb</code>	CDB Holder	Investimento em CDB	20	Comum
<code>poupador_dedicado</code>	Poupador Dedicado	Total investido ≥ R\$10.000	75	Épico
<code>milionario</code>	Rumo ao Milhão	Total investido ≥ R\$100.000	100	Lendário

Código	Título	Como desbloquear	Pontos	Raridade
investidor_diversificado	Diversificado	3+ tipos de investimento	60	Épico

Grupo: Hábitos

Código	Título	Como desbloquear	Pontos	Raridade
disciplinado	Disciplinado	10 despesas pagas no mesmo mês	30	Raro
poupador	Poupador	Poupou $\geq 20\%$ da renda em 1 mês	40	Raro
lembrete_mestre	Mestre dos Lembretes	5 lembretes cadastrados	20	Comum
automatico	Piloto Automático	1ª recorrência configurada	25	Comum
orcamentista	Orçamentista	1º orçamento por categoria criado	25	Comum
tagger	Organizador (Tags)	Tags usadas em 5 despesas	25	Comum
analista	Analista Financeiro	Consultou o relatório IA	30	Raro
comparador	Analítico	Consultou o comparativo mensal	20	Comum

Grupo: Reservas

Código	Título	Como desbloquear	Pontos	Raridade
reserva_iniciada	Reserva Iniciada	Criou a reserva de emergência legado	25	Comum
reserva_1_mes	Reserva: 1 Mês	Reserva cobre 1+ mês	30	Raro
reserva_3_meses	Reserva: 3 Meses	Reserva cobre 3+ meses	60	Épico
reserva_6_meses	Reserva: 6 Meses	Reserva cobre 6+ meses	80	Épico
reserva_completa	Reserva Completa!	100% da meta atingida	100	Lendário
multi_reserva_criada	Organizado por Natureza	1ª reserva especializada	30	Comum
multi_3_reservas	Mestre das Reservas	3 reservas ativas	80	Épico

Código	Título	Como desbloquear	Pontos	Raridade
reserva_spec_completa	Objetivo Alcançado!	Reserva especializada completada	100	Lendário

Grupo: Metas

Código	Título	Como desbloquear	Pontos	Raridade
meta_concluida	Conquistador	Concluiu a 1ª meta	50	Épico
meta_casa	Sonho da Casa Própria	Meta de imóvel criada	50	Épico
meta_carro	Sobre Rodas	Meta de veículo criada	40	Raro
meta_viagem	Explorador	Meta de viagem criada	30	Raro
meta_educacao	Investidor no Futuro	Meta de educação criada	30	Raro
meta_liberdade	Liberdade Financeira	Meta de independência criada	75	Épico
meta_aposentadoria	Aposentado Tranquilo	Meta de aposentadoria criada	75	Épico

Grupo: Dívidas

Código	Título	Como desbloquear	Pontos	Raridade
primeiro_carro	Nas Rodas!	Empréstimo de veículo	30	Raro
primeiro_imovel	Proprietário!	1º financiamento imobiliário	50	Épico
quitou_10pct	Primeiros 10%	10% do financiamento pago	30	Raro
quitou_15pct	Conquistando Espaço	15% do financiamento pago	40	Raro
fatura_paga	Fatura Quitada	Pagou fatura completa do cartão	30	Comum
sem_dividas	Livre de Dívidas!	Quitou todas as dívidas	200	Lendário
amortizou_simulou	Estrategista da Dívida	Usou o simulador de amortização	30	Raro

Grupo: v3.0

Código	Título	Como desbloquear	Pontos	Raridade
sub_detector_sca nned	Detetive Financeiro	Escaneou assinaturas	20	Comum
sub_cancelou_1	Economizador	Cancelou 1 assinatura	40	Raro
desafio_52_inici ou	52 Semanas Aceito	Iniciou o desafio	25	Comum
desafio_52_metad e	Na Metade do Caminho	26 semanas completadas	75	Épico
desafio_52_compl eto	Campeão das 52 Semanas	52 semanas completadas	200	Lendário
regra_503020_ver de	Equilíbrio Financeiro	Score 50/30/20 $\geq$ 80	60	Épico

Como as conquistas são desbloqueadas

O sistema usa `INSERT OR IGNORE` — se a conquista já existe para o usuário, não insere novamente:

```
await db.prepare(
  'INSERT OR IGNORE INTO conquistas_usuario (user_id, conquista_codigo, visualizado) VALUES
  (?, ?, 0)'
).bind(userId, codigo).run()
```

O campo `visualizado = 0` sinaliza ao frontend que deve exibir o toast/notificação.

GET /api/conquistas

Retorna todas as conquistas do catálogo com flag `desbloqueada: true/false`, pontos totais e percentual de completude.

30. CDI EM TEMPO REAL

Rota: GET /api/cdi — `src/routes/cdi.ts`

Fluxo

- Consulta `cdi_historico` para a taxa mais recente
- Se dados < 24h → retorna cache
- Se desatualizado → busca na API pública do BCB (Banco Central)
- Converte taxa diária → mensal → anual
- Salva em `cdi_historico`

Uso pelos demais módulos

- **Investimentos** → `cdi_atual` salvo em cada investimento tipo caixinha
- **Simulador** → taxa dinâmica para simulações de caixinha

- **Dashboard** → widget de CDI atual

31. PERFIL DO USUÁRIO

Rota: /api/perfil — src/routes/perfil.ts

Atualizar perfil

```
PUT /api/perfil {  
  nome?: string  
  avatar_color?: string // cor hex do avatar  
  perfil_investidor?: string // conservador | moderado | arrojado | agressivo  
}
```

Alterar senha

```
PUT /api/perfil/senha { senha_atual, nova_senha }  
→ Verifica senha atual via Web Crypto  
→ Hash da nova senha  
→ Atualiza em users
```

Excluir conta

```
DELETE /api/perfil { confirmar: 'EXCLUIR' }  
→ Deleta usuário em cascata (sessions, receitas, despesas, metas, etc.)  
→ IRREVERSÍVEL
```

32. PAINEL ADMINISTRATIVO

Rota: /admin — src/routes/admin.ts

Acesso: Basic Auth com ADMIN_PASSWORD (variável de ambiente)

Funcionalidades

- Dashboard de estatísticas: total de usuários, distribuição por plano, registros por tabela
- Explorador de tabelas: visualização paginada de qualquer tabela do banco
- Console SQL: executar queries diretamente no D1 (SELECT apenas em produção)
- Gerenciamento de usuários: visualizar, alterar plano, resetar senha

33. MAPA DE INTERDEPENDÊNCIAS

■ ALIMENTAM O DASHBOARD ■

■ Receitas → Despesas → Investimentos → Metas → Empréstimos ■
 ■ Financiamentos → Cartões (card_charges) ■

■ MÓDULOS QUE LEEM DESPESAS ■

■ Dashboard · Orçamentos · Regra 50/30/20 · Comparativo · IA ■
 ■ Relatório · Detector Assinaturas · Tags · Alertas Cartão ■

■ MÓDULOS QUE LEEM RECEITAS ■

■ Dashboard · Regra 50/30/20 · Comparativo · IA · Projeção ■
 ■ Relatório · Simulador Amortização (fluxo de caixa) ■

■ MÓDULOS DEPENDENTES DE FINANCIAMENTOS/EMPRÉSTIMOS ■

■ Dashboard (comprometimento) · Metas debt_payoff ■
 ■ Simulador Amortização · IA · Relatório ■

■ MÓDULOS QUE SOMAM DADOS NA REGRA 50/30/20 ■

■ Receitas (renda base) · Despesas (classificadas por categoria) ■
 ■ Investimentos (somam a Poupança) · Reservas Esp. (somam a Poupança) ■

■ MÓDULOS QUE GERAM CONQUISTAS ■

■ Receitas · Despesas · Investimentos · Metas · Cartões ■
 ■ Reservas (legado + especializadas) · Lembretes · Recorrências ■
 ■ Financiamentos · Empréstimos · IA · Desafio 52 · Assinaturas ■
 ■ Regra 50/30/20 · Amortização · Tags · Comparativo ■

Cadeia crítica: Despesas → Cartões → Fatura

Despesa com cartao_id

```
■
■■■ Cria card_charge (billing_month, billing_year, status=pendente)
■■■ Decrementa limite_disponivel do cartão
■
■■■ Ao pagar (PATCH /despesas/:id/status)
■■■ Atualiza despesas.status = 'pago'
■■■ Atualiza card_charges.status = 'pago'
■■■ Restaura limite_disponivel
```

Cadeia crítica: Financiamento → Meta → Sincronização

```
Financiamento cadastrado
■
■■■ Gera despesas futuras automaticamente
■■■ Alimenta Dashboard com saldo_devedor e parcela_mensal
■
■■■ Meta categoria='debt_payoff'
■
■■■ POST /metas/sincronizar-dividas
■ ■■■ Lê saldo_devedor atual do financiamento
■ ■■■ Atualiza valor_atual da meta
■
■■■ Se saldo = 0 → Meta 'concluida' + conquista 'sem_dividas'
```

34. FLUXO DE DADOS COMPLETO

Ciclo mensal típico de um usuário

```
1. INÍCIO DO MÊS
■■■ Recorrências pendentes aparecem com botão "Gerar Agora"
■■■ Lembretes urgentes aparecem no badge do sidebar
■■■ Orçamentos resetam (são por mês)

2. DURANTE O MÊS
■■■ Registra Receitas (salário, freelance, etc.)
■■■ Registra Despesas
■ ■■■ À vista → status='pago', data=hoje
■ ■■■ Pendente → status='pendente', vencimento=data_de_vencimento
■ ■■■ Cartão → cria card_charge na fatura correta
■
■■■ Dashboard recalcula em tempo real
■ ■■■ Score de Saúde (Premium)
■ ■■■ Comprometimento de renda
■ ■■■ Vencimentos próximos
■
■■■ Orçamentos monitoram gastos por categoria automaticamente

3. FIM DO MÊS
■■■ Paga Despesas pendentes (PATCH /status)
■■■ Paga Fatura do Cartão → restaura limite
■■■ Verifica Assinaturas Fantasma (POST /assinaturas-fantasma/scan)
■■■ Analisa Regra 50/30/20 (score de equilíbrio)
■■■ Deposita em Reservas Especializadas
■■■ Marca semanas do Desafio 52 Semanas

4. ANÁLISE ANUAL
■■■ Relatório Anual (Premium) → 12 meses consolidados
■■■ Projeção Financeira → tendência para os próximos 6 meses
■■■ IA / Diagnóstico 360° → análise completa com recomendações
```

35. TODOS OS ENDPOINTS DA API

Autenticação

Método	Endpoint	Auth	Descrição
GET	/api/auth/check-email?email=X	Não	Validar e-mail em tempo real
POST	/api/auth/register	Não	Criar conta
POST	/api/auth/verify-otp	Não	Verificar código OTP
POST	/api/auth/resend-otp	Não	Reenviar código OTP
POST	/api/auth/login	Não	Login
POST	/api/auth/logout	Sim	Logout
GET	/api/auth/me	Sim	Dados do usuário logado

Dashboard

Método	Endpoint	Auth	Plano
GET	/api/dashboard	Sim	Todos
GET	/api/dashboard/relatorio?ano=X	Sim	Premium+

Receitas

Método	Endpoint	Descrição
GET	/api/receitas	Listar (filtros: mes, ano, categoria)
POST	/api/receitas	Criar receita
PUT	/api/receitas/:id	Editar receita
DELETE	/api/receitas/:id	Excluir receita
GET	/api/receitas/categorias	Totais por categoria

Despesas

Método	Endpoint	Descrição
GET	/api/despesas	Listar (filtros: mes, ano, categoria, status)
POST	/api/despesas	Criar (com parcelamento e cartão)
PUT	/api/despesas/:id	Editar
PATCH	/api/despesas/:id/status	Mudar status (pago/pendente)

Método	Endpoint	Descrição
DELETE	/api/despesas/:id	Excluir
GET	/api/despesas/categorias	Totais por categoria

Cartões

Método	Endpoint	Descrição
GET	/api/cartoes	Listar cartões com uso calculado
POST	/api/cartoes	Cadastrar cartão
PUT	/api/cartoes/:id	Editar cartão
DELETE	/api/cartoes/:id	Desativar cartão
GET	/api/cartoes/:id/fatura	Faturas do cartão
POST	/api/cartoes/:id/pagar-fatura	Pagar fatura completa
POST	/api/cartoes/:id/lancamento	Lançar compra no cartão

Metas

Método	Endpoint	Descrição
GET	/api/metass	Listar metas
POST	/api/metass	Criar meta
PUT	/api/metass/:id	Editar meta
PATCH	/api/metass/:id/deposito	Adicionar progresso
DELETE	/api/metass/:id	Excluir meta
POST	/api/metass/sincronizar-dividas	Atualizar metas debt_payoff

Orçamentos

Método	Endpoint	Descrição
GET	/api/orçamentos?mes=X&ano=X	Listar orçamentos com gasto real
POST	/api/orçamentos	Criar orçamento
PUT	/api/orçamentos/:id	Editar orçamento
DELETE	/api/orçamentos/:id	Excluir orçamento

Recorrências

Método	Endpoint	Descrição
GET	/api/recorrencias	Listar recorrências
POST	/api/recorrencias	Criar recorrência (Premium+)
PUT	/api/recorrencias/:id	Editar recorrência
DELETE	/api/recorrencias/:id	Excluir recorrência
POST	/api/recorrencias/:id/gerar	Gerar lançamento do mês

Lembretes

Método	Endpoint	Descrição
GET	/api/lembretes	Listar lembretes com urgência
POST	/api/lembretes	Criar lembrete
PUT	/api/lembretes/:id	Editar lembrete
PATCH	/api/lembretes/:id/status	Marcar como pago/ignorado
DELETE	/api/lembretes/:id	Excluir lembrete

Investimentos

Método	Endpoint	Descrição
GET	/api/investimentos	Listar com rendimento recalculado
POST	/api/investimentos	Cadastrar investimento
PUT	/api/investimentos/:id	Editar investimento
DELETE	/api/investimentos/:id	Excluir investimento
GET	/api/investimentos/simulacao	Simular rendimento (Premium+)

Reserva Legado

Método	Endpoint	Descrição
GET	/api/reserva	Buscar reserva com métricas
POST	/api/reserva	Criar reserva
PUT	/api/reserva/:id	Atualizar reserva
DELETE	/api/reserva/:id	Excluir reserva

Reservas Especializadas (v3.0)

Método	Endpoint	Descrição
GET	/api/reservas-esp	Listar reservas com resumo

Método	Endpoint	Descrição
POST	/api/reservas-esp	Criar reserva especializada
PUT	/api/reservas-esp/:id	Editar reserva
DELETE	/api/reservas-esp/:id	Excluir reserva
POST	/api/reservas-esp/:id/depositar	Depositar valor
POST	/api/reservas-esp/:id/sacar	Sacar valor
GET	/api/reservas-esp/:id/historico	Histórico de transações

Financiamentos

Método	Endpoint	Descrição
GET	/api/financiamentos	Listar financiamentos
POST	/api/financiamentos	Cadastrar financiamento
PUT	/api/financiamentos/:id	Editar financiamento
POST	/api/financiamentos/:id/pagar	Registrar pagamento de parcelas
DELETE	/api/financiamentos/:id	Excluir financiamento

Empréstimos

Método	Endpoint	Descrição
GET	/api/emprestimos	Listar empréstimos
POST	/api/emprestimos	Cadastrar empréstimo
PUT	/api/emprestimos/:id	Editar empréstimo
POST	/api/emprestimos/:id/pagar	Registrar pagamento
DELETE	/api/emprestimos/:id	Excluir empréstimo

Amortização (v3.0)

Método	Endpoint	Descrição
POST	/api/amortizacao/simular	Simular amortização (Premium+)
GET	/api/amortizacao/historico	Histórico de simulações

Assinaturas Fantasma (v3.0)

Método	Endpoint	Descrição
GET	/api/assinaturas-fantasma	Listar detectadas
POST	/api/assinaturas-fantasma/scan	Escanear despesas
PATCH	/api/assinaturas-fantasma/:id/feedback	Dar feedback

Regra 50/30/20 (v3.0)

Método	Endpoint	Descrição
GET	/api/regra-503020?mes=X&ano=X	Análise do período

Desafio 52 Semanas (v3.0)

Método	Endpoint	Descrição
GET	/api/desafio-52?ano=X	Buscar (cria as 52 semanas se necessário)
PATCH	/api/desafio-52/:semana?ano=X	Atualizar status da semana
POST	/api/desafio-52/reset?ano=X	Reiniciar todas as semanas

Análises

Método	Endpoint	Descrição
GET	/api/projecao	Projeção 3 e 6 meses (Premium+)
GET	/api/comparativo?...	Comparativo entre meses
GET	/api/relatorio	Relatório avançado
GET	/api/ia/insights	Análise IA 360° (Premium+)
GET	/api/ia/diagnostico-360	Diagnóstico completo (Premium+)
GET	/api/cdi	CDI real atual
GET	/api/conquistas	Listar conquistas

Tags e Alertas

Método	Endpoint	Descrição
GET	/api/tags	Listar tags
POST	/api/tags	Criar tag
DELETE	/api/tags/:id	Excluir tag

Método	Endpoint	Descrição
GET	/api/alertas-cartao	Listar alertas de cartão
POST	/api/alertas-cartao	Configurar alerta

Perfil e Admin

Método	Endpoint	Descrição
GET	/api/perfil	Dados do perfil
PUT	/api/perfil	Atualizar perfil
PUT	/api/perfil/senha	Alterar senha
DELETE	/api/perfil	Excluir conta
GET	/admin	Painel admin (Basic Auth)
GET	/api/health	Health check

Documentação gerada automaticamente a partir do código-fonte em 13/03/2026.

VerdeMais v3.0 — 9.816 linhas de frontend, 30 rotas de backend, 35 tabelas de banco de dados.

Documentação gerada automaticamente a partir do código-fonte em 13/03/2026.

VerdeMais v3.0 — 9.816 linhas de frontend · 30 rotas de backend · 35 tabelas de banco de dados.